

# Technical Requirement Document — Time-Off Microservice

## 1. Executive Summary

ExampleHR needs a dedicated Time-Off Microservice responsible for managing time-off request lifecycle, exposing accurate employee balances, and maintaining balance integrity while integrating with an external Human Capital Management system, such as Workday or SAP.

The HCM remains the source of truth for employment and balance data. ExampleHR is the employee-facing system where employees request time off and managers approve those requests. Because balances may change independently in the HCM — for example, annual refreshes, work anniversary grants, manual corrections, or policy-driven adjustments — the microservice must be designed defensively.

The proposed solution is a NestJS-based backend service using SQLite for local persistence. It exposes REST APIs for employees, managers, and system integrations; integrates with HCM via real-time APIs and batch imports; maintains a local balance projection; validates requests before and during approval; and includes reconciliation, idempotency, auditability, and robust automated tests.

REST is recommended over GraphQL for this take-home assignment because the domain is workflow-driven, command-oriented, integration-heavy, and easier to test and reason about with explicit resources and status transitions. GraphQL is evaluated as an alternative but rejected for this scope.

---

## 2. Product Context

ExampleHR provides the primary interface for employees to request time off. However, the HCM system remains the authoritative source of truth for employment data and official balances.

The main challenge is maintaining consistency between ExampleHR's local view and HCM's authoritative balance state. The employee expects instant feedback and accurate balances. The manager needs confidence that approvals are based on valid and current data.

Balances are assumed to be maintained per employee and per location.

Example:

An employee has 10 days of leave for location X. They request 2 days in ExampleHR. ExampleHR must validate that the balance is available, prevent duplicate or conflicting submissions, and ensure that the HCM accepts the request. Later, the HCM may independently update the balance due to a work anniversary bonus or yearly refresh. ExampleHR must detect, import, reconcile, and expose the updated balance.

## 3. Goals

### 3.1 Functional Goals

1. Allow employees to view available time-off balances by employee and location.
2. Allow employees to create, cancel, and view time-off requests.
3. Allow managers to approve or reject pending requests.
4. Validate time-off requests against known local balances before submission.
5. Validate requests with HCM before or during final approval.
6. Submit approved time-off requests to HCM.
7. Import full balance corpus from HCM batch endpoints.
8. Support real-time HCM balance lookup.
9. Reconcile local projected balances against HCM authoritative data.
10. Maintain full request lifecycle history and audit trail.
11. Prevent duplicate requests and duplicate HCM submissions.
12. Handle invalid dimensions, insufficient balances, stale balances, HCM downtime, and inconsistent HCM responses.

### 3.2 Non-Functional Goals

1. Reliability: The service must be resilient to temporary HCM failures.
  2. Data integrity: No request should incorrectly consume balance twice.
  3. Observability: Important operations should be logged and traceable.
  4. Testability: The system must include rigorous automated tests and mock HCM endpoints.
  5. Maintainability: The codebase should be modular and aligned with NestJS best practices.
  6. Security: APIs should enforce authorization and prevent employee/manager privilege violations.
  7. Idempotency: Repeated client or integration calls must not create duplicate side effects.
  8. Defensive design: HCM errors are expected but not blindly trusted as the only safeguard.
- 

## 4. Non-Goals

1. Building a full HR policy engine.
  2. Supporting complex accrual policy calculation locally.
  3. Replacing HCM as the source of truth.
  4. Supporting payroll calculations.
  5. Supporting multiple types of leave policies beyond generic time-off types unless represented as dimensions.
  6. Building a production-grade distributed database architecture.
  7. Implementing real authentication providers; test authentication may be mocked.
  8. Building frontend UI.
- 

## 5. Key Assumptions

1. HCM is the authoritative source of truth for official balances.

2. ExampleHR may keep a local balance projection for fast reads and UX responsiveness.
  3. Balances are per employee and per location.
  4. HCM exposes:
    5. A real-time API to get balance for an employee/location combination.
    6. A real-time API to submit or file time-off usage.
    7. A batch endpoint or feed containing the complete corpus of balances.
    8. HCM may reject invalid employee/location combinations.
    9. HCM may reject requests with insufficient balance.
  10. HCM error responses are useful but not guaranteed to be complete or always reliable.
  11. SQLite is required by the assignment and will be used with transactional discipline.
  12. NestJS is required by the assignment.
  13. Mock HCM endpoints will be implemented as part of the automated test suite.
- 

## 6. Recommended API Style: REST

### 6.1 Decision

Use REST APIs.

### 6.2 Rationale

REST is the better fit for this assignment because the system is centered around clear resources and lifecycle transitions:

- Time-off requests
- Balances
- Employees
- Locations
- HCM sync jobs
- Reconciliation results

The domain also has command-like operations such as submit, approve, reject, cancel, sync, and reconcile. These are easier to expose, secure, test, and audit through explicit REST endpoints.

REST also simplifies:

1. Idempotency handling with headers such as `Idempotency-Key`.
2. HTTP semantics for status codes.
3. Contract testing.
4. Integration with mock HCM servers.
5. Observability and debugging.
6. Take-home assignment evaluation.

## 6.3 Alternative: GraphQL

GraphQL could be useful if ExampleHR needed a highly flexible frontend data composition layer where clients query deeply nested employee, manager, request, and balance views.

However, GraphQL is not recommended for this specific service because:

1. The most important operations are commands with side effects.
2. Strong workflow boundaries are more important than flexible client-driven queries.
3. GraphQL mutations would still need the same transactional and idempotency protections.
4. REST endpoints are easier to validate with E2E tests and mock integrations.
5. REST better communicates integration contracts for HCM-facing workflows.

## 6.4 Final Position

REST should be used for the Time-Off Microservice. GraphQL could later be added as a read-only API gateway layer if ExampleHR needs frontend aggregation, but it should not be the primary interface for this service.

---

# 7. High-Level Architecture

## 7.1 Components

### 1. API Layer

2. NestJS controllers.
3. REST endpoints.
4. Request validation.
5. Authentication and authorization guards.
6. Idempotency handling.

### 7. Application Service Layer

8. Time-off request lifecycle orchestration.
9. Balance validation.
10. Approval workflow.
11. Cancellation workflow.
12. HCM sync coordination.

### 13. Domain Layer

14. Request status model.

15. Balance projection rules.

16. Validation rules.

17. State transition rules.

#### **18. Persistence Layer**

19. SQLite database.

20. Repository classes.

21. Transactions.

22. Outbox/inbox tables.

23. Audit log.

#### **24. HCM Integration Client**

25. Real-time balance lookup.

26. Real-time time-off submission.

27. Batch balance import.

28. Retry handling.

29. Timeout handling.

30. Response mapping.

#### **31. Sync and Reconciliation Jobs**

32. Batch import processor.

33. Balance reconciliation.

34. Failed submission retry.

35. Outbox event dispatcher.

#### **36. Mock HCM Server**

37. Simulates balance reads.

38. Simulates time-off submission.

39. Simulates insufficient balance.

40. Simulates invalid dimensions.

41. Simulates delayed or inconsistent balance updates.

42. Simulates HCM downtime and transient failures.

## 8. Domain Model

### 8.1 Time-Off Request Lifecycle

Supported statuses:

1. DRAFT
2. SUBMITTED
3. PENDING\_MANAGER\_APPROVAL
4. APPROVED\_PENDING\_HCM
5. APPROVED
6. REJECTED
7. CANCELLED
8. FAILED\_HCM\_VALIDATION
9. FAILED\_HCM\_SUBMISSION
10. RECONCILIATION\_REQUIRED

### 8.2 State Transition Rules

Allowed transitions:

- DRAFT → SUBMITTED
- SUBMITTED → PENDING\_MANAGER\_APPROVAL
- PENDING\_MANAGER\_APPROVAL → REJECTED
- PENDING\_MANAGER\_APPROVAL → APPROVED\_PENDING\_HCM
- APPROVED\_PENDING\_HCM → APPROVED
- APPROVED\_PENDING\_HCM → FAILED\_HCM\_VALIDATION
- APPROVED\_PENDING\_HCM → FAILED\_HCM\_SUBMISSION
- SUBMITTED → CANCELLED
- PENDING\_MANAGER\_APPROVAL → CANCELLED
- APPROVED → CANCELLED , only if HCM cancellation succeeds or the cancellation is marked pending HCM sync
- Any terminal or inconsistent state → RECONCILIATION\_REQUIRED , only through reconciliation logic

Invalid examples:

- REJECTED cannot become APPROVED .
  - CANCELLED cannot become APPROVED .
  - APPROVED cannot be deleted.
  - A request cannot be approved twice.
  - A failed HCM submission cannot be silently marked approved without successful retry or explicit reconciliation.
-

## 9. Balance Model

### 9.1 Balance Dimensions

Balances are stored per:

- `employeeId`
- `locationId`

Optional future dimensions may include:

- `timeOffType`
- `policyId`
- `effectiveDate`
- `unit`, such as days or hours

For the assignment, the required model is employee plus location.

### 9.2 Balance Types

The service should distinguish between:

#### 1. Authoritative HCM Balance

2. Last known balance received from HCM.

3. Imported via batch or real-time lookup.

#### 4. Local Projected Balance

5. HCM balance minus locally pending or approved requests not yet reflected in HCM.

6. Used for fast UX feedback.

#### 7. Reserved Balance

8. Balance temporarily reserved by submitted or pending approval requests.

#### 9. Available Balance

10. Balance available to the employee after subtracting active reservations.

### 9.3 Balance Integrity Rule

A time-off request should only proceed when:

```
requestedAmount <= availableBalance
```

However, final approval must still validate with HCM because the HCM may have changed independently.

---

## 10. Data Model

### 10.1 employees

| Column          | Type        | Notes                    |
|-----------------|-------------|--------------------------|
| id              | TEXT PK     | Internal employee ID     |
| hcm_employee_id | TEXT UNIQUE | External HCM employee ID |
| manager_id      | TEXT NULL   | Manager relationship     |
| status          | TEXT        | ACTIVE, INACTIVE         |
| created_at      | DATETIME    | Creation timestamp       |
| updated_at      | DATETIME    | Update timestamp         |

### 10.2 locations

| Column          | Type        | Notes                    |
|-----------------|-------------|--------------------------|
| id              | TEXT PK     | Internal location ID     |
| hcm_location_id | TEXT UNIQUE | External HCM location ID |
| name            | TEXT        | Location name            |
| created_at      | DATETIME    | Creation timestamp       |
| updated_at      | DATETIME    | Update timestamp         |

### 10.3 balances

| Column            | Type    | Notes                          |
|-------------------|---------|--------------------------------|
| id                | TEXT PK | Balance row ID                 |
| employee_id       | TEXT FK | Employee                       |
| location_id       | TEXT FK | Location                       |
| hcm_balance       | NUMERIC | Last HCM known balance         |
| reserved_balance  | NUMERIC | Locally reserved amount        |
| available_balance | NUMERIC | hcm_balance - reserved_balance |

| Column           | Type          | Notes                                  |
|------------------|---------------|----------------------------------------|
| unit             | TEXT          | DAYS by default                        |
| hcm_version      | TEXT NULL     | HCM version or checksum when available |
| last_hcm_sync_at | DATETIME NULL | Last sync timestamp                    |
| created_at       | DATETIME      | Creation timestamp                     |
| updated_at       | DATETIME      | Update timestamp                       |

Unique constraint:

`employee_id + location_id`

Check constraints:

- `hcm_balance >= 0`
- `reserved_balance >= 0`
- `available_balance >= 0`, unless temporarily marked for reconciliation

#### 10.4 `time_off_requests`

| Column             | Type      | Notes               |
|--------------------|-----------|---------------------|
| id                 | TEXT PK   | Request ID          |
| employee_id        | TEXT FK   | Request owner       |
| location_id        | TEXT FK   | Balance location    |
| amount             | NUMERIC   | Requested days      |
| unit               | TEXT      | DAYS                |
| start_date         | DATE      | Start date          |
| end_date           | DATE      | End date            |
| status             | TEXT      | Lifecycle status    |
| manager_id         | TEXT NULL | Approving manager   |
| hcm_transaction_id | TEXT NULL | HCM confirmation ID |
| idempotency_key    | TEXT NULL | Client idempotency  |
| failure_reason     | TEXT NULL | Failure reason      |
| created_at         | DATETIME  | Creation timestamp  |
| updated_at         | DATETIME  | Update timestamp    |

Unique constraint:

`employee_id + idempotency_key`, when idempotency key is provided.

## 10.5 `request_status_history`

| Column      | Type      | Notes                     |
|-------------|-----------|---------------------------|
| id          | TEXT PK   | History row               |
| request_id  | TEXT FK   | Time-off request          |
| from_status | TEXT NULL | Previous status           |
| to_status   | TEXT      | New status                |
| actor_type  | TEXT      | EMPLOYEE, MANAGER, SYSTEM |
| actor_id    | TEXT NULL | Actor ID                  |
| reason      | TEXT NULL | Reason                    |
| created_at  | DATETIME  | Timestamp                 |

## 10.6 `hcm_sync_jobs`

| Column        | Type          | Notes                                                     |
|---------------|---------------|-----------------------------------------------------------|
| id            | TEXT PK       | Job ID                                                    |
| type          | TEXT          | BATCH_IMPORT, REALTIME_LOOKUP, SUBMISSION, RECONCILIATION |
| status        | TEXT          | PENDING, RUNNING, COMPLETED, FAILED                       |
| started_at    | DATETIME NULL | Start timestamp                                           |
| completed_at  | DATETIME NULL | Completion timestamp                                      |
| error_message | TEXT NULL     | Error                                                     |
| created_at    | DATETIME      | Creation timestamp                                        |

## 10.7 `outbox_events`

| Column         | Type    | Notes                     |
|----------------|---------|---------------------------|
| id             | TEXT PK | Event ID                  |
| aggregate_type | TEXT    | TIME_OFF_REQUEST, BALANCE |

| Column        | Type          | Notes                 |
|---------------|---------------|-----------------------|
| aggregate_id  | TEXT          | Related entity        |
| event_type    | TEXT          | Event name            |
| payload       | TEXT          | JSON payload          |
| status        | TEXT          | PENDING, SENT, FAILED |
| retry_count   | INTEGER       | Retry count           |
| next_retry_at | DATETIME NULL | Retry timestamp       |
| created_at    | DATETIME      | Creation timestamp    |

## 10.8 `audit_logs`

| Column      | Type      | Notes         |
|-------------|-----------|---------------|
| id          | TEXT PK   | Audit log ID  |
| entity_type | TEXT      | Entity type   |
| entity_id   | TEXT      | Entity ID     |
| action      | TEXT      | Action        |
| actor_type  | TEXT      | Actor type    |
| actor_id    | TEXT NULL | Actor ID      |
| metadata    | TEXT      | JSON metadata |
| created_at  | DATETIME  | Timestamp     |

# 11. API Design

## 11.1 Employee APIs

### Get Employee Balances

`GET /employees/:employeeId/balances`

Returns balances for the authenticated employee.

Response:

```
{
  "employeeId": "emp_123",
  "balances": [
    {
      "locationId": "loc_001",
      "hcmBalance": 10,
      "reservedBalance": 2,
      "availableBalance": 8,
      "unit": "DAYS",
      "lastHcmSyncAt": "2026-01-01T10:00:00Z"
    }
  ]
}
```

Rules:

- Employee can only view their own balances.
- Manager can view balances of direct reports.
- Admin/system role can view all.

---

## Real-Time Balance Refresh

```
POST /employees/:employeeId/locations/:locationId/balances/refresh
```

Purpose:

Fetch latest balance from HCM real-time API and update local projection.

Response:

```
{
  "employeeId": "emp_123",
  "locationId": "loc_001",
  "hcmBalance": 12,
  "reservedBalance": 2,
  "availableBalance": 10,
  "source": "HCM_REALTIME",
  "lastHcmSyncAt": "2026-01-01T10:00:00Z"
}
```

Rules:

- Must preserve local reservations.

- Must not blindly overwrite pending local state.
- If HCM returns invalid dimensions, local balance should be marked unavailable or reconciliation-required.

---

## Create Time-Off Request

POST /time-off-requests

Headers:

Idempotency-Key: <client-generated-key>

Request:

```
{
  "employeeId": "emp_123",
  "locationId": "loc_001",
  "amount": 2,
  "unit": "DAYS",
  "startDate": "2026-02-10",
  "endDate": "2026-02-11"
}
```

Response:

```
{
  "requestId": "tor_001",
  "status": "PENDING_MANAGER_APPROVAL",
  "availableBalanceAfterReservation": 8
}
```

Rules:

1. Validate employee/location combination exists locally.
2. Validate amount is positive.
3. Validate dates are valid.
4. Validate requested amount against local available balance.
5. Reserve balance transactionally.
6. Create request transactionally.
7. Return the existing request if the same idempotency key is reused.
8. Do not call HCM submission yet; final HCM submission happens on approval.

Failure cases:

- 400 Bad Request for invalid payload.
  - 403 Forbidden for unauthorized employee.
  - 409 Conflict for insufficient local balance.
  - 422 Unprocessable Entity for invalid business rule.
  - 503 Service Unavailable only if the operation requires HCM and HCM is unavailable.
- 

### Get Time-Off Request

```
GET /time-off-requests/:requestId
```

Returns request details, current status, status history, and HCM transaction information when available.

---

### Cancel Time-Off Request

```
POST /time-off-requests/:requestId/cancel
```

Rules:

1. Employee can cancel own request if not terminal.
  2. If request is not yet approved in HCM, release local reservation.
  3. If request is already approved in HCM, submit cancellation to HCM or mark cancellation pending.
  4. Add audit log and status history.
- 

## 11.2 Manager APIs

### List Pending Requests

```
GET /managers/:managerId/time-off-requests?status=PENDING_MANAGER_APPROVAL
```

Rules:

- Manager can only view direct reports unless admin role.
- 

### Approve Request

```
POST /time-off-requests/:requestId/approve
```

Headers:

Idempotency-Key: <manager-action-key>

Flow:

1. Validate manager is authorized.
2. Load request with transactional lock.
3. Confirm request is in `PENDING_MANAGER_APPROVAL`.
4. Refresh or validate balance with HCM real-time API.
5. If HCM balance is insufficient, mark `FAILED_HCM_VALIDATION`, release local reservation, and return conflict.
6. If HCM validates, submit time-off to HCM.
7. If HCM accepts, mark request `APPROVED`, persist HCM transaction ID, and update local balance projection.
8. If HCM fails transiently, mark `FAILED_HCM_SUBMISSION` or keep `APPROVED_PENDING_HCM` depending on failure type and retry strategy.
9. Write status history and audit log.

Response:

```
{
  "requestId": "tor_001",
  "status": "APPROVED",
  "hcmTransactionId": "hcm_tx_789"
}
```

Important defensive rule:

Approval is not complete until the HCM submission succeeds or the system explicitly records the request as pending HCM submission with retry.

---

## Reject Request

POST /time-off-requests/:requestId/reject

Request:

```
{
  "reason": "Business coverage conflict"
}
```

Rules:

1. Validate manager authorization.

2. Move request to `REJECTED`.
  3. Release reserved balance.
  4. Write status history and audit log.
- 

## 11.3 System APIs

### Batch Balance Import

`POST /system/hcm/balances/batch-import`

Purpose:

Import full HCM balance corpus.

Flow:

1. Create sync job.
2. Fetch batch data from HCM mock endpoint.
3. Validate payload structure.
4. Upsert employees, locations, and balances.
5. Preserve local reservations.
6. Recalculate available balances.
7. Detect conflicts:
  8. Local reserved balance greater than new HCM balance.
  9. Unknown employee/location dimensions.
  10. Requests pending for balances that no longer exist.
11. Mark affected requests or balances as `RECONCILIATION_REQUIRED`.
12. Store sync job result.

Response:

```
{
  "jobId": "job_001",
  "status": "COMPLETED",
  "importedBalances": 1000,
  "reconciliationRequired": 3
}
```

---

## Reconciliation

POST /system/reconciliation/run

Purpose:

Compare local projected balances against HCM authoritative state and identify inconsistencies.

Examples of inconsistencies:

1. Local balance differs from HCM after accounting for reservations.
2. Approved local request not found in HCM.
3. HCM balance lower than local reserved balance.
4. Employee/location combination invalid in HCM but still active locally.
5. HCM accepted request but local status did not update due to network timeout.

Response:

```
{
  "jobId": "recon_001",
  "status": "COMPLETED",
  "issues": [
    {
      "type": "LOCAL_RESERVED_EXCEEDS_HCM_BALANCE",
      "employeeId": "emp_123",
      "locationId": "loc_001",
      "severity": "HIGH"
    }
  ]
}
```

---

## Health Check

GET /health

Response:

```
{
  "status": "ok",
  "database": "ok",
  "hcmMock": "ok"
}
```

---

## 12. HCM Mock API

The test suite must include realistic HCM mock endpoints.

### 12.1 Get Real-Time Balance

GET /mock-hcm/employees/:employeeId/locations/:locationId/balance

Success:

```
{
  "employeeId": "emp_123",
  "locationId": "loc_001",
  "balance": 10,
  "unit": "DAYS",
  "version": "v10"
}
```

Errors:

- 404 invalid employee/location combination.
- 409 dimension conflict.
- 500 transient HCM error.
- Delayed response timeout scenario.

---

### 12.2 Submit Time-Off Usage

POST /mock-hcm/time-off

Request:

```
{
  "employeeId": "emp_123",
  "locationId": "loc_001",
  "amount": 2,
  "unit": "DAYS",
  "externalRequestId": "tor_001"
}
```

Success:

```
{
  "transactionId": "hcm_tx_789",
  "status": "ACCEPTED",
  "remainingBalance": 8
}
```

Errors:

- 400 invalid dimensions.
- 409 insufficient balance.
- 409 duplicate external request ID.
- 500 transient failure.
- Timeout after accepting request, to test reconciliation and idempotent retry.

---

### 12.3 Batch Balance Corpus

GET /mock-hcm/balances/batch

Response:

```
{
  "balances": [
    {
      "employeeId": "emp_123",
      "locationId": "loc_001",
      "balance": 10,
      "unit": "DAYS",
      "version": "v10"
    }
  ]
}
```

Scenarios:

1. Normal full corpus.
  2. Work anniversary bonus.
  3. Start-of-year refresh.
  4. Manual HCM adjustment.
  5. Removed employee/location combination.
  6. Corrupted or partial payload.
  7. Duplicate rows.
  8. Same employee across multiple locations.
-

## 13. Consistency and Failure Handling

### 13.1 Local-First UX With HCM Validation

The employee should get fast feedback based on local projected balances. However, final approval must validate with HCM before completing.

This balances user experience with correctness.

### 13.2 Transaction Boundaries

The following operations must be transactional:

1. Creating request and reserving balance.
2. Rejecting request and releasing reservation.
3. Cancelling request and releasing reservation.
4. Approving request local state transition.
5. Applying batch balance updates.

### 13.3 Idempotency

All write endpoints must support idempotency where duplicate requests are likely:

- Create time-off request.
- Approve request.
- Reject request.
- Cancel request.
- Submit to HCM.
- Batch import job trigger.

Client-provided `Idempotency-Key` must be stored and reused to return the same result.

HCM submissions must include `externalRequestId` so HCM can reject or return duplicate submissions deterministically.

### 13.4 Retry Strategy

Transient HCM errors should be retried with exponential backoff.

Non-retryable errors:

- Invalid dimensions.
- Insufficient balance.
- Employee not found.
- Location not found.

Retryable errors:

- Timeout.
- HTTP 500.
- HTTP 502.
- HTTP 503.
- HTTP 504.
- Network failure.

### 13.5 Timeout After HCM Accepts

A critical failure mode occurs when ExampleHR submits to HCM, HCM accepts the request, but ExampleHR times out before receiving confirmation.

Mitigation:

1. Use `externalRequestId` in HCM submission.
2. Retry submission idempotently.
3. If HCM returns duplicate, fetch transaction state.
4. If still uncertain, mark request `RECONCILIATION_REQUIRED`.
5. Do not release local reservation until resolved.

### 13.6 Batch Import Conflicts

If batch import shows that HCM balance is lower than locally reserved balance:

1. Do not silently overwrite local state.
2. Mark affected balance as reconciliation required.
3. Mark affected pending requests as requiring review.
4. Prevent new requests for that employee/location until resolved or refreshed.

---

## 14. Security and Authorization

### 14.1 Roles

1. `EMPLOYEE`
2. `MANAGER`
3. `SYSTEM_ADMIN`
4. `SYSTEM_INTEGRATION`

### 14.2 Rules

1. Employees can only view and create requests for themselves.
2. Managers can only approve or reject requests for direct reports.
3. System APIs require system integration role.
4. HCM mock endpoints should not be publicly exposed in production.

5. Audit logs must record actor type and actor ID.
  6. All write operations must validate authorization before state changes.
- 

## 15. Observability

### 15.1 Logs

Log key events:

- Request created.
- Balance reserved.
- Request approved.
- HCM submission started.
- HCM submission succeeded.
- HCM submission failed.
- Batch import started/completed/failed.
- Reconciliation issue detected.

### 15.2 Metrics

Recommended metrics:

- Number of requests created.
- Number of approvals.
- HCM submission latency.
- HCM error rate.
- Batch import duration.
- Reconciliation issue count.
- Failed HCM submissions.
- Idempotency conflict count.

### 15.3 Tracing

Use correlation IDs across:

- API request.
  - Database transaction.
  - HCM call.
  - Outbox event.
  - Reconciliation job.
- 

## 16. Test Strategy

Because the assignment emphasizes agentic development and test rigor, the test suite is a first-class deliverable.

## 16.1 Unit Tests

Coverage targets:

- Domain state transitions.
- Balance calculation.
- Request validation.
- Idempotency behavior.
- HCM error mapping.
- Authorization guards.
- Date validation.
- Retry classification.
- Reconciliation rule evaluation.

Examples:

1. Request amount greater than available balance should fail.
2. Pending request should reserve balance.
3. Rejected request should release balance.
4. Approved request should not be approved twice.
5. Invalid transition should throw domain error.
6. HCM insufficient balance should map to conflict.
7. Timeout should be retryable.
8. Invalid dimensions should be non-retryable.

## 16.2 Integration Tests

Integration tests should run with SQLite and NestJS testing module.

Required cases:

1. Create request successfully and reserve balance.
2. Create duplicate request with same idempotency key returns same request.
3. Reject request releases balance.
4. Manager approval calls HCM and marks approved.
5. HCM insufficient balance prevents approval and releases reservation.
6. Batch import updates balances while preserving reservations.
7. Batch import detects reserved balance greater than HCM balance.
8. Reconciliation detects local/HCM mismatch.
9. Unauthorized manager cannot approve request.
10. Employee cannot access another employee's request.

## 16.3 End-to-End Tests

E2E tests should start:

1. Time-Off Microservice.
2. SQLite test database.

3. Mock HCM server.

Required flows:

### **Happy Path**

1. Seed employee, manager, location, and balance.
2. Employee views balance.
3. Employee creates request.
4. Manager approves.
5. Service submits to HCM.
6. Request becomes approved.
7. Balance is updated.

### **Insufficient Balance at Request Time**

1. Employee has 1 day available.
2. Employee requests 2 days.
3. API returns conflict.
4. No request is created.
5. Balance is unchanged.

### **Insufficient Balance at Approval Time**

1. Employee has 10 days locally.
2. Employee requests 2 days.
3. HCM balance changes independently to 1 day.
4. Manager approves.
5. HCM rejects.
6. Request becomes failed HCM validation.
7. Local reservation is released.
8. Balance is refreshed.

### **Work Anniversary Bonus**

1. Employee has 10 days.
2. HCM batch sends 12 days.
3. Local balance is updated.
4. Available balance reflects new HCM balance minus reservations.

### **Timeout After HCM Accepts**

1. Approval submits to HCM.
2. HCM accepts but mock server times out.
3. Service retries with same external request ID.
4. HCM returns duplicate or existing transaction.
5. Service marks request approved without double-consuming balance.

### Invalid Employee/Location Dimension

1. Employee creates request for local employee/location.
2. HCM later says the combination is invalid.
3. Approval fails.
4. Request is marked failed validation.
5. Audit log is created.

### 16.4 Contract Tests

Contract tests should validate that the HCM client correctly handles:

1. Successful balance response.
2. Successful submission response.
3. Insufficient balance error.
4. Invalid dimensions error.
5. Timeout.
6. Malformed response.
7. Duplicate submission response.
8. Batch corpus response.

### 16.5 Regression Tests

Regression tests should protect against:

1. Double approval.
2. Double reservation.
3. Negative balances.
4. Lost reservation after batch import.
5. Unauthorized approval.
6. Incorrect idempotency handling.
7. Local state becoming approved without HCM confirmation.

### 16.6 Suggested Coverage Thresholds

Minimum recommended thresholds:

```
{
  "branches": 85,
  "functions": 90,
  "lines": 90,
  "statements": 90
}
```

Critical domain modules should target near 100% coverage:

- Balance calculation.

- State transitions.
  - HCM error mapping.
  - Reconciliation rules.
- 

## 17. Agentic Development Instructions

The agent should not write code until the TRD and test cases are accepted.

Recommended implementation order:

1. Create NestJS project structure.
2. Implement domain models and state transition unit tests first.
3. Implement SQLite schema and repositories.
4. Implement balance service.
5. Implement time-off request service.
6. Implement REST controllers.
7. Implement HCM client interface.
8. Implement mock HCM server.
9. Implement approval workflow.
10. Implement batch import.
11. Implement reconciliation job.
12. Implement audit logging.
13. Implement E2E tests.
14. Generate coverage report.
15. Refactor only after tests are green.

Agent guardrails:

1. Do not skip tests.
  2. Do not implement business logic inside controllers.
  3. Do not call HCM directly from controllers.
  4. Do not update balance outside a transaction.
  5. Do not approve a request without HCM validation.
  6. Do not silently swallow HCM failures.
  7. Do not allow negative available balance unless explicitly marked reconciliation-required.
  8. Do not create multiple requests for the same idempotency key.
  9. Do not use GraphQL for the primary API.
  10. Keep code modular and readable.
- 

## 18. Acceptance Criteria

### 18.1 Functional Acceptance Criteria

1. Employees can view their balances.

2. Employees can create time-off requests.
3. Requests reserve local balance.
4. Managers can approve or reject pending requests.
5. Approval validates with HCM.
6. HCM insufficient balance blocks approval.
7. Batch import updates balances.
8. Batch import preserves local reservations.
9. Reconciliation identifies mismatches.
10. Duplicate client calls do not create duplicate requests or submissions.
11. Audit history is available for each request.
12. Mock HCM endpoints support happy path and failure scenarios.

## **18.2 Technical Acceptance Criteria**

1. Built with NestJS.
  2. Uses SQLite.
  3. REST API is documented.
  4. Test suite includes unit, integration, E2E, and contract tests.
  5. Coverage report is generated.
  6. Critical domain logic has high coverage.
  7. HCM mock server is part of the test suite.
  8. Database migrations or schema setup are repeatable.
  9. Application can be run locally with clear instructions.
  10. Repository includes README with architecture, setup, test instructions, and trade-off analysis.
- 

## **19. Alternatives Considered**

### **19.1 REST vs GraphQL**

REST was selected because the service is resource-oriented, command-heavy, easier to test, and more suitable for workflow and integration boundaries. GraphQL was rejected for the primary API because it adds complexity without solving the core consistency problem.

### **19.2 Local Projection vs Always Query HCM**

Always querying HCM would provide freshness but would harm UX, increase latency, increase HCM dependency, and reduce resilience.

Local projection was selected because it provides fast reads and supports offline resilience. However, it must be validated with HCM during approval and reconciled through batch imports.

### **19.3 Submit to HCM at Request Creation vs Manager Approval**

Submitting to HCM at request creation would reserve or consume official HCM balance before manager approval, which may be incorrect.

Submitting at manager approval was selected because HCM should only receive final approved requests. Local reservations protect the user experience before approval.

## 19.4 SQLite vs Production Database

SQLite is required by the assignment and is acceptable for a take-home project. In production, PostgreSQL would be recommended for stronger concurrency, locking, operational tooling, and scalability.

## 19.5 Synchronous Approval vs Async Approval

Synchronous approval gives immediate manager feedback but depends on HCM availability. Async approval improves resilience but creates more pending states.

Recommended approach:

- Try synchronous HCM validation and submission.
- If transient failure occurs, mark as pending or failed HCM submission depending on the exact failure.
- Retry safely using idempotency and reconciliation.

## 20. Risks and Mitigations

| Risk                                             | Impact | Mitigation                                                     |
|--------------------------------------------------|--------|----------------------------------------------------------------|
| HCM balance changes after local request creation | High   | Validate with HCM during approval                              |
| HCM timeout after accepting submission           | High   | Use externalRequestId and idempotent retry                     |
| Batch import overwrites local reservations       | High   | Preserve reservations and recalculate projection               |
| Double approval                                  | High   | Transactional state transition and idempotency                 |
| Negative balances                                | High   | Check constraints and reconciliation rules                     |
| Invalid dimensions                               | Medium | Validate locally and defensively handle HCM rejection          |
| Unauthorized manager approval                    | High   | Authorization guards and manager relationship checks           |
| Mock HCM too simplistic                          | Medium | Include failure modes, delays, duplicates, malformed responses |
| SQLite write concurrency                         | Medium | Keep transactions short and test concurrent requests           |

| Risk                            | Impact | Mitigation                                |
|---------------------------------|--------|-------------------------------------------|
| Agent-generated code regression | High   | Strong test suite and coverage thresholds |

## 21. Open Questions

1. Does HCM support idempotent submission using an external request ID?
2. Does HCM expose transaction lookup by external request ID?
3. Are balances always in days, or can they be in hours?
4. Are partial days supported?
5. Can employees request time off across multiple locations?
6. Can a request span weekends or holidays, and should those days count?
7. Does HCM support cancellation or reversal?
8. Are managers sourced from HCM or ExampleHR?
9. Should batch import be pull-based, push-based, or both?
10. What is the expected maximum corpus size for batch import?
11. Are there multiple HCM providers per customer?
12. Is multi-tenancy required?

## 22. Final Recommendation

Build the Time-Off Microservice as a NestJS REST service backed by SQLite, with a local balance projection, transactional request lifecycle management, HCM real-time validation, batch balance import, reconciliation, idempotent write operations, audit logging, and rigorous automated tests.

The design should prioritize correctness, defensive integration behavior, and regression protection over feature breadth. The most important engineering principle is that ExampleHR may provide fast local feedback, but HCM remains the final authority before approval is completed.

The implementation should be considered successful only when the test suite proves that the system prevents double-consumption, handles stale balances, survives HCM failures, preserves reservations during batch imports, and detects reconciliation issues.